

Week 1:

Topic: Data Structures And Algorithm

Exercise 1: Inventory Management System

Step 1:-

A warehouse is basically a storage facility or a place where enormous amount of goods and items are stored, managed and distributed before they are purchased by their respective customers. Thus, to efficiently manage all the goods and items we will be using various types of data structures and algorithms.

Data Structure and Algorithm help by:-

- Providing fast searching for the products.
- Providing quick insertion and deletion.
- Reducing memory usage.
- Making the application scalable for a huge number of inventories.
- Improving the overall performance of the application.

Example: If a warehouse consists of 10000 of products, then it will be a very difficult task to search for a provide one by one as it will be very time consuming. Thus, here with the help of a Data Structure called HashMap searching will become very easy and efficient as its time complexity for searching is $O(1)$, i.e. , constant.

Data Structures that can be used are:-

- ArrayList: As the size of the ArrayList is not fixed which will help us to insert the products in an efficient manner without any space constraint. It is very easy is manage and its:
 - § Insertion TC: $O(1)$
 - § Deletion TC: $O(1)$ -best case and $O(n)$ - worst case
 - § Searching TC: $O(1)$ - best case and $O(n)$ – worst case
- LinkedList: Size is not fixed, i.e , its is also a dynamic data structure.
 - § Insertion TC: $O(1)$ – at beginning and $O(n)$ – at end or at any index
 - § Deletion TC: $O(1)$ – at beginning and $O(n)$ – at end and at any index
 - § Searching TC: $O(n)$

- HashMap: Best Data Structure to use in the problem statement as it provides $O(1)$ TC for searching and is very easy to update and maintain.
- TreeMap : It also provides a sorted arrangement of all the items thus making it easy to manage and access.

Step 2 and 3:-

```

J inventoryManagementSystem.java X
1  import java.util.*;
2
3  class Product
4  {
5      int productId;
6      String productName;
7      int quantity;
8      double price;
9
10     Product(int productId , String productName , int quantity , double price)    // Constructor
11     {
12         this.productId = productId;
13         this.productName = productName;
14         this.quantity = quantity;
15         this.price = price;
16     }
17
18     public String toString()
19     {
20         return productId + " " + productName + "Qty:" + quantity + "Price:" + price;
21     }
22 }

```

```

23 public class inventoryManagementSystem
24 {
25     static HashMap<Integer , Product> inventory = new HashMap<>();
26
27     static void addProduct(Product p)
28     {
29         inventory.put(p.productId , p);
30     }
31
32     static void updateProduct(int id , int quantity , double price)
33     {
34         if(inventory.containsKey(id))
35         {
36             Product p = inventory.get(id);
37             p.quantity = quantity;
38             p.price = price;
39         }
40     }
41
42     static void displayProducts()
43     {
44         for(Product p : inventory.values())
45         {
46             System.out.println(p);
47         }
48     }
49
50     static void deleteProduct(int id)
51     {
52         inventory.remove(id);
53     }
54 }

```

```

55 Run | Debug
56 public static void main(String args[])
57 {
58     addProduct(new Product(productId: 101, productName: "Laptop", quantity: 100, price: 50000));
59     addProduct(new Product(productId: 102, productName: "Mouse", quantity: 20, price: 8000));
60     addProduct(new Product(productId: 103, productName: "TV", quantity: 3500, price: 65000));
61
62     updateProduct(id: 101, quantity: 105, price: 620000);
63
64     displayProducts();
65
66     deleteProduct(id: 103);
67
68     System.out.println(x: "\nAfter deletion:");
69     displayProducts();
70 }
71 }
72

```

Step 4:-

Analysis of the different functions:

Add: $O(1)$

Update: $O(1)$

Delete: $O(1)$

Search: $O(1)$

Optimization:

Use of HashMap instead of ArrayList will improve its efficiency.

Exercise 2: E-commerce Platform Search Function

- Understanding the Asymptotic Notations:

Big O notation is a mathematical way to describe how the running time or memory usage of an algorithm grows as the input size increases. It is used to find the upper bound for an algorithm.

Instead of measuring execution time in seconds, it measures how the number of operations grow relative to the input size n .

Examples:

- 1) $O(1)$ – Constant time
- 2) $O(\log n)$ – Logarithmic time
- 3) $O(n)$ – Linear time
- 4) $O(n \log n)$ – Efficient sorting algorithm

5) $O(n^2)$ – Nested loops

Time Complexity comparison for searching operations:

- 1) Linear Search: Best – $O(1)$, Average – $O(n)$, Worst – $O(n)$
 - 2) Binary Search: Best – $O(1)$, Average – $O(\log n)$, Worst – $O(\log n)$
- Step up:



The screenshot shows a Java IDE with three tabs: `inventoryManagementSystem.java 1`, `Product.java 4` (active), and `product2.java`. The `Product.java` file contains the following code:

```
1  import java.util.*;
2
3  class Product
4  {
5      int productId;
6      String productName;
7      String category;
8
9      Product(int id,String name,String category){
10         this.productId=id;
11         this.productName=name;
12         this.category=category;
13     }
14 }
```

- Implementation of linear and binary search:

Linear Search:

```

16 class LinearSearch
17 {
18
19     static int search(Product arr[],String key){
20
21         for(int i=0;i<arr.length;i++){
22
23             if(arr[i].productName.equals(key))
24                 return i;
25
26         }
27
28         return -1;
29     }
30
31     Run main | Debug main | Run | Debug
32     public static void main(String args[]){
33
34         Product arr[]={
35             new Product(id: 1,name: "Laptop",category: "Electronics"),
36             new Product(id: 2,name: "Mouse",category: "Electronics"),
37             new Product(id: 3,name: "Shoes",category: "Fashion")
38         };
39
40         System.out.println(search(arr,key: "Mouse"));
41     }
42 }

```

Binary Search:

```

class BinarySearch {
    static int search(Product arr[],String key){
        int low=0;
        int high=arr.length-1;

        while(low<=high){
            int mid=(low+high)/2;

            int cmp=arr[mid].productName.compareTo(key);

            if(cmp==0)
                return mid;

            if(cmp<0)
                low=mid+1;
            else
                high=mid-1;
        }

        return -1;
    }

    Run main | Debug main | Run | Debug
    public static void main(String args[]){
        Product arr[]={
            new Product(id: 1,name: "Laptop",category: "Electronics"),
            new Product(id: 2,name: "Mouse",category: "Electronics"),
            new Product(id: 3,name: "Shoes",category: "Fashion")
        };

        System.out.println(search(arr,key: "Mouse"));
    }
}

```

- Analysis:

Time Complexity:

Linear Search: Best - $O(1)$, Average - $O(n)$, Worst - $O(n)$

Binary Search: Best - $O(1)$, Average - $O(\log n)$, Worst - $O(\log n)$

Here ,binary search is preferred because it has a time complexity of $O(\log n)$, making it much faster than Linear Search's $O(n)$ for large datasets. It repeatedly divides the search space into two halves, significantly reducing the number of comparisons. However, Binary Search can only be used when the data is sorted. Therefore, it is the ideal choice for searching products in a large, sorted e-commerce inventory.

Exercise 3 : Sorting Customer Orders

- Step 1: Understanding Sorting Algorithms

Bubble Sort: It is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element moves to its correct position at the end of the array. It is easy to implement but inefficient for large datasets because it performs many comparisons and swaps. Its average and worst-case time complexity is **$O(n^2)$** .

Insertion Sort: It builds a sorted portion of the array one element at a time by inserting each new element into its correct position. It is efficient for small or nearly sorted datasets because it performs fewer shifts when the data is already partially ordered. The algorithm is simple, stable, and requires no extra memory. Its average and worst-case time complexity is **$O(n^2)$** .

Quick Sort: It is a divide-and-conquer algorithm that selects a pivot element and partitions the array into two parts based on the pivot. The same process is applied recursively until the array is completely sorted. It is one of the fastest sorting algorithms for large datasets due to its efficient partitioning strategy. Its average time complexity is **$O(n \log n)$** , while the worst case is **$O(n^2)$** .

Merge Sort: Merge Sort is a divide-and-conquer algorithm that recursively divides the array into smaller halves, sorts each half, and then merges them into a sorted array. It guarantees consistent performance regardless of the input order and is a stable sorting algorithm. However, it requires additional memory for the merging process. Its best, average, and worst-case time complexity is **$O(n \log n)$** .

- Step 2: Stepup

```

1  import java.util.*;
2
3  class Order{
4
5      int orderId;
6      String customerName;
7      double totalPrice;
8
9      Order(int id,String name,double price){
10
11          orderId=id;
12          customerName=name;
13          totalPrice=price;
14
15      }
16
17  }

```

- Step 3: Implementation

Bubble Sort:-

```

19  class BubbleSort {
20
21      static void bubble(Order arr[]){
22
23          int n=arr.length;
24
25          for(int i=0;i<n-1;i++){
26
27              for(int j=0;j<n-i-1;j++){
28
29                  if(arr[j].totalPrice>arr[j+1].totalPrice){
30
31                      Order temp=arr[j];
32                      arr[j]=arr[j+1];
33                      arr[j+1]=temp;
34                  }
35              }
36          }
37      }
38  }

```

Quick Sort:-

```

40 class QuickSort {
41
42     static int partition(Order arr[],int low,int high){
43
44         double pivot=arr[high].totalPrice;
45
46         int i=low-1;
47
48         for(int j=low;j<high;j++){
49
50             if(arr[j].totalPrice<pivot){
51
52                 i++;
53
54                 Order temp=arr[i];
55                 arr[i]=arr[j];
56                 arr[j]=temp;
57             }
58         }
59         Order temp=arr[i+1];
60         arr[i+1]=arr[high];
61         arr[high]=temp;
62
63         return i+1;
64     }
65
66     static void quick(Order arr[],int low,int high){
67
68         if(low<high){
69
70             int pi=partition(arr,low,high);
71             quick(arr,low,pi-1);
72             quick(arr,pi+1,high);
73         }
74     }
75 }

```

- Step 4: Analysis

Bubble Sort and Quick Sort differ significantly in terms of performance. Bubble Sort has a best-case time complexity of $O(n)$ when the array is already sorted, but its average and worst-case time complexity is $O(n^2)$ because it repeatedly compares and swaps adjacent elements. In contrast, Quick Sort has an average-case time complexity of $O(n \log n)$, making it much faster for large datasets, although its worst-case time complexity is $O(n^2)$ when the pivot selection is poor. Therefore, Quick Sort performs considerably better than Bubble Sort in most practical situations.

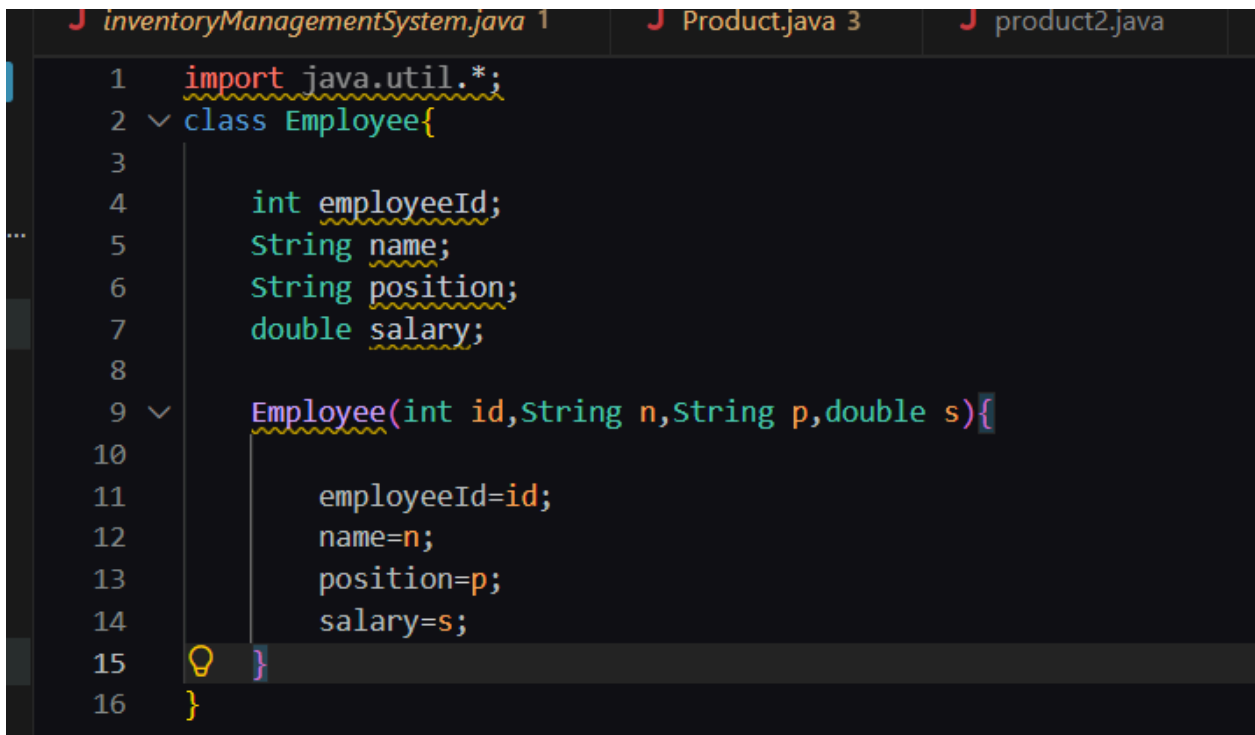
Quick Sort is generally preferred over Bubble Sort because it is much more efficient for sorting large datasets. It uses the **divide-and-conquer** approach, which reduces the number of comparisons and swaps required to sort the elements. As a result, its average time complexity of **$O(n \log n)$** makes it significantly faster than Bubble Sort's **$O(n^2)$** average complexity. Although Quick Sort has a worst-case complexity of **$O(n^2)$** , choosing a good pivot usually avoids this situation, making it the preferred sorting algorithm in real-world applications.

Exercise 4: Employee Management System

Step 1: Understand Array representation

An array is a collection of elements of the same data type stored in contiguous (continuous) memory locations. Each element occupies an equal amount of memory, and the address of any element can be calculated using its index, allowing direct access. This contiguous storage makes arrays highly efficient for accessing and processing data. The main advantages of arrays are fast random access ($O(1)$), simple implementation, efficient memory usage, and suitability for storing a fixed number of elements.

Step 2: Stepout



```
1  import java.util.*;
2  class Employee{
3
4      int employeeId;
5      String name;
6      String position;
7      double salary;
8
9      Employee(int id,String n,String p,double s){
10
11          employeeId=id;
12          name=n;
13          position=p;
14          salary=s;
15      }
16  }
```

The screenshot shows a code editor with three tabs: 'inventoryManagementSystem.java 1', 'Product.java 3', and 'product2.java'. The 'inventoryManagementSystem.java' tab is active, displaying the following Java code for the 'Employee' class. The code includes an import statement for 'java.util.*', a class declaration 'class Employee{', and a constructor 'Employee(int id,String n,String p,double s){'. The constructor body assigns the parameters to instance variables: 'employeeId=id;', 'name=n;', 'position=p;', and 'salary=s;'. The class is closed with '}'.

Step 3: Implementation

```

18 class EmployeeArray {
19
20     Employee arr[]=new Employee[10];
21     int size=0;
22
23     void add(Employee e)
24     {
25         arr[size++]=e;
26     }
27
28     void traverse()
29     {
30         for(int i=0;i<size;i++)
31             System.out.println(arr[i].name);
32     }
33
34     Employee search(int id)
35     {
36         for(int i=0;i<size;i++)
37             if(arr[i].employeeId==id)
38                 return arr[i];
39         return null;
40     }
41
42     void delete(int id)
43     {
44         int index=-1;
45         for(int i=0;i<size;i++)
46             if(arr[i].employeeId==id)
47                 index=i;
48
49         if(index!=-1){
50             for(int i=index;i<size-1;i++)
51                 arr[i]=arr[i+1];
52             size--;
53         }
54     }
55 }

```

Step 4: Analysis

The time complexity of array operations depends on how the elements are accessed or modified. Adding an element at the end of the array takes $O(1)$ time if space is available, while inserting at a specific position requires shifting elements, resulting in $O(n)$ time. Searching for an element requires checking each element sequentially, so its time complexity is $O(n)$. Traversing the array to access all elements also takes $O(n)$ time. Deleting an element requires shifting the remaining elements to fill the gap, giving it a time complexity of $O(n)$.

Limitations: Arrays have a fixed size, which means their capacity cannot be changed after creation. Inserting or deleting elements is inefficient because existing elements need to be shifted, resulting in $O(n)$ time complexity. Arrays also require contiguous memory, which may be difficult to allocate for very large arrays. They are best used when the number of elements is known in advance, frequent random access is required, and the data size does not change often.

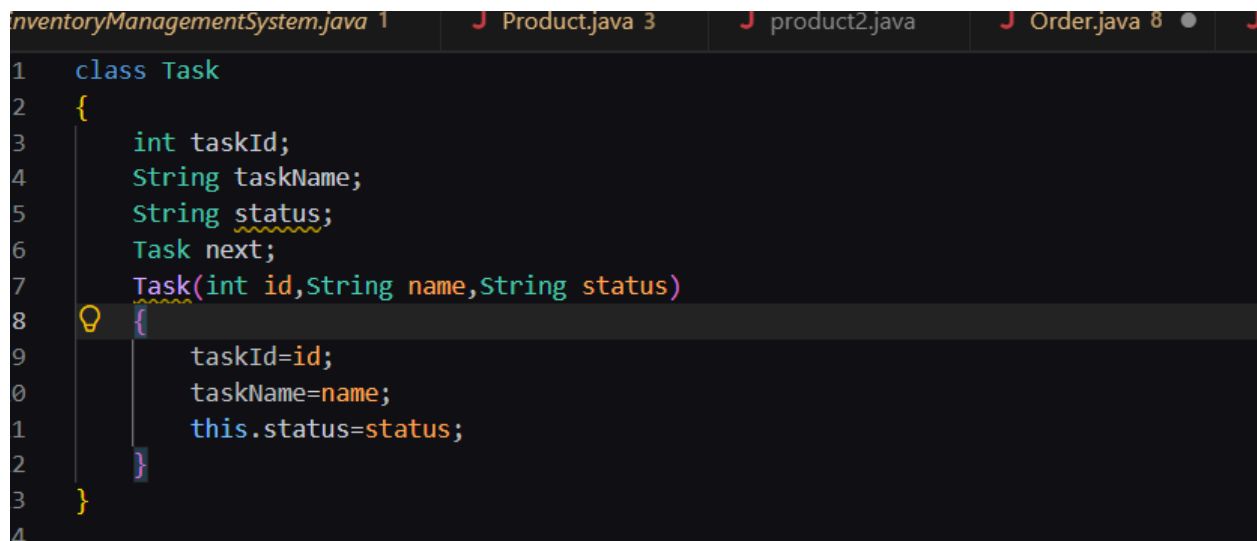
Exercise 5: Task Management System

Step 1: Understand LinkedList

Singly LinkedList: A Singly Linked List is a linear data structure in which each node contains two parts: data and a pointer (next) to the next node in the sequence. Traversal is possible only in the forward direction, starting from the head node until the last node. It requires less memory than a doubly linked list because each node stores only one pointer. It is suitable for applications where forward traversal is sufficient.

Doubly LinkedList: A Doubly Linked List is a linear data structure in which each node contains data, a pointer to the previous node (prev), and a pointer to the next node (next). It allows traversal in both forward and backward directions, making insertion and deletion operations more efficient when the node is known. However, it requires additional memory to store the extra pointer. It is commonly used in applications such as browser history, undo/redo functionality, and navigation systems.

Step 2: Setup

A screenshot of a code editor with a dark theme. The editor shows a Java file named 'Task.java' with the following code:

```
1 class Task
2 {
3     int taskId;
4     String taskName;
5     String status;
6     Task next;
7     Task(int id,String name,String status)
8     {
9         taskId=id;
10        taskName=name;
11        this.status=status;
12    }
13 }
14
```

The code is color-coded: 'class' is blue, 'int', 'String', and 'Task' are green, and 'id', 'name', 'status', and 'this' are orange. The 'next' variable is in blue. The constructor parameters are in orange. The code is enclosed in curly braces. The editor has a tab bar at the top with 'Task.java' selected. There are also tabs for 'Product.java', 'product2.java', and 'Order.java'. A yellow lightbulb icon is visible next to the constructor definition.

Step 3: Implementation

```

15 class TaskList
16 {
17     Task head;
18
19     void add(Task t)
20     {
21         if(head==null)
22         {
23             head=t;
24             return;
25         }
26         Task temp=head;
27         while(temp.next!=null)
28             temp=temp.next;
29         temp.next=t;
30     }
31
32     void traverse()
33     {
34         Task temp=head;
35         while(temp!=null)
36         {
37             System.out.println(temp.taskName);
38             temp=temp.next;
39         }
40     }
41

```

```

42     void search(int id)
43     {
44         Task temp=head;
45         while(temp!=null)
46         {
47             if(temp.taskId==id)
48             {
49                 System.out.println(x: "Found");
50                 return;
51             }
52             temp=temp.next;
53         }
54     }
55
56     void delete(int id)
57     {
58         if(head.taskId==id)
59         {
60             head=head.next;
61             return;
62         }
63         Task temp=head;
64         while(temp.next!=null)
65         {
66             if(temp.next.taskId==id)
67             {
68                 temp.next=temp.next.next;
69                 return;
70             }
71             temp=temp.next;
72         }
73     }
74 }

```

Step 4: Analysis

Time Complexity of each operation: The time complexity of operations in a Singly Linked List depends on traversing the list to locate the required node. Adding a node at the beginning takes $O(1)$ time, while adding at the end takes $O(n)$ if no tail pointer is maintained. Searching for a node requires traversing the list, resulting in $O(n)$ time complexity. Similarly, traversing the entire list takes $O(n)$, and deleting a node also requires searching for it first, giving an overall time complexity of $O(n)$.

Advantages: Linked lists are more suitable for dynamic data because they can grow or shrink in size during program execution without requiring a fixed capacity. Unlike arrays, linked lists do not require contiguous memory allocation, making memory usage more flexible. Insertion and deletion operations are more efficient because elements do not need to be shifted after modifications. Therefore, linked lists are preferred in applications where the size of the data changes frequently and frequent insertions or deletions are required.

Exercise 6: Library Management System

Step 1: Understanding the Search Algorithms

Linear Search: It is a simple searching algorithm that checks each element of the list one by one until the required element is found or the end of the list is reached. It works on both sorted and unsorted data and is easy to implement. However, it becomes inefficient for large datasets because it may need to examine every element. Its time complexity is $O(n)$ in the average and worst cases.

Binary Search: Binary Search is an efficient searching algorithm that works only on sorted data. It repeatedly compares the target element with the middle element of the array and eliminates half of the remaining search space in each step. This process continues until the element is found or the search space becomes empty. Its time complexity is $O(\log n)$, making it much faster than Linear Search for large sorted datasets.

Step 2: Setup

```
public class Book
{
    int bookId;
    String title;
    String author;
    Book(int id,String t,String a)
    {
        bookId=id;
        title=t;
        author=a;
    }
}
```

Step 3: Implementation

Linear Search:

```
14 static Book linear(Book arr[],String key)
15 {
16     for(Book b:arr)
17         if(b.title.equals(key))
18             return b;
19     return null;
20 }
21
```

Binary Search:

```

13
14  static Book linear(Book arr[],String key)
15  {
16      for(Book b:arr)
17          if(b.title.equals(key))
18              return b;
19      return null;
20  }
21
22  static Book binary(Book arr[],String key)
23  {
24      int low=0;
25      int high=arr.length-1;
26      while(low<=high)
27      {
28          int mid=(low+high)/2;
29          int cmp=arr[mid].title.compareTo(key);
30          if(cmp==0)
31              return arr[mid];
32          if(cmp<0)
33              low=mid+1;
34          else
35              high=mid-1;
36      }
37      return null;
38  }
39

```

Step 4: Analysis

Time Complexity: Linear Search checks each element one by one until the target is found, so its time complexity is $O(n)$ in the average and worst cases. Binary Search repeatedly divides the search space into two halves, giving it a much faster time complexity of $O(\log n)$. Therefore, Binary Search is significantly more efficient than Linear Search for large datasets, provided the data is sorted.

Linear Search should be used when the data is unsorted, the dataset is small, or the list changes frequently, as it does not require sorting. Binary Search should be used when the data is sorted and the dataset is large, since it quickly reduces the search space by half in each step. Thus, Linear Search is simple and flexible, while Binary Search is the preferred choice for fast searching in large, sorted datasets.

Exercise 7: Financial Forecasting

Step 1: Understand Recursive Algorithm

Recursion is a programming technique in which a function calls itself to solve a problem by breaking it into smaller, similar subproblems. Every recursive function has a base case that stops the recursion and a recursive case that continues calling the function with a smaller input. Recursion simplifies problems that have a repetitive or hierarchical structure, such as factorial calculation, Fibonacci series, tree traversal, and divide-and-conquer algorithms. It often makes the code shorter, easier to understand, and closer to the mathematical definition of the problem.

Step 2 and 3 : Setup and Implementation

```
1 public class Forecast {  
2  
3     static double predict(double value,double rate,int years){  
4  
5         if(years==0)  
6             return value;  
7  
8         return predict(value*(1+rate),rate,years-1);  
9  
10    }  
11  
12    Run main | Debug main | Run | Debug  
13    public static void main(String args[]){  
14  
15        System.out.println(predict(value: 10000,rate: 0.10,years: 5));  
16    }  
17  
18 }
```

Step 4: Analysis

Time Complexity: In the Financial Forecasting exercise, the recursive algorithm computes the future value by making one recursive call for each time period. Since each recursive call performs a constant amount of work, the algorithm has a time complexity of $O(n)$, where n is the number of years or periods. The recursion also uses $O(n)$ auxiliary space because of the function call stack.

The recursive solution can be optimized by using Memoization or Dynamic Programming, which store the results of previously computed recursive calls and reuse them instead of recalculating them. Another optimization is to replace recursion with an iterative approach, which eliminates the overhead of recursive function calls and reduces space complexity to $O(1)$. These techniques improve performance and prevent excessive computation, especially for large input values.

